

Efficient Hyperparameter Optimization for an English-to-Hindi Transformer

via Ray Tune + Optuna with ASHA

Abhay Kashyap (B22CS001)

CSL7120: DLOps — Indian Institute of Technology Jodhpur

Abstract

This report documents an end-to-end study of English-to-Hindi neural machine translation using a custom Transformer implemented in PyTorch. First, a strong baseline is established by training the model for 100 epochs and evaluating with NLTK’s corpus-level BLEU. Next, the training loop is refactored to become compatible with Ray Tune, enabling scalable hyperparameter search. Ray Tune is configured to use Optuna as the underlying search algorithm, while the ASHA scheduler is employed to terminate underperforming trials early, thereby improving computational efficiency. Finally, the best-performing hyperparameters from the search are used to retrain the model and obtain the final tuned BLEU score.

1 Introduction

Neural machine translation (NMT) benefits significantly from appropriate hyperparameter selection, as model quality and convergence behavior are highly sensitive to learning rate, batch size, attention configuration, and regularization. In this assignment, we optimize a Transformer-based sequence-to-sequence model for English-to-Hindi translation using:

- **Ray Tune** for distributed and repeatable hyperparameter tuning,
- **Optuna** as a Bayesian optimization method for smarter exploration,
- **ASHA** (Asynchronous Successive Halving Algorithm) for early stopping.

The central objective is to achieve at least the baseline BLEU score while constraining each Ray Tune trial to a maximum number of epochs to reduce overall computation.

2 Dataset and Preprocessing

The dataset consists of English–Hindi sentence pairs stored in `English-Hindi.tsv`. Each row contains an English sentence and its corresponding Hindi translation.

2.1 Data cleaning

The dataset is loaded into a dataframe, the columns `en` and `hi` are retained, missing values are removed, and the index is reset.

2.2 Vocabulary construction

Token-level vocabularies are constructed for both languages using a simple frequency thresholding scheme. Tokens with frequency lower than the threshold are mapped to `<unk>`. Special tokens include: `<pad>`, `<sos>`, `<eos>`, and `<unk>`.

2.3 Sequence encoding

Each sentence is encoded into a fixed-length sequence with padding. The decoder input and target output sequences are created by shifting the Hindi token sequence:

- decoder input: Hindi tokens excluding `<eos>`
- decoder output: Hindi tokens excluding `<sos>`

3 Transformer Model Architecture

The model is a custom Transformer encoder–decoder with the following configurable components:

- Model dimension `d_model`
- Number of attention heads `num_heads` (with the constraint that `d_model` is divisible by `num_heads`)
- Feed-forward dimension `d_ff`
- Number of encoder/decoder layers `num_layers`
- Dropout rate `dropout`

3.1 Training objective

Training uses cross-entropy loss with the target padding index ignored:

$$\mathcal{L} = \text{CE}(\text{logits}, \text{targets}), \quad \text{ignoring } \text{<pad>}.$$

This loss serves as the optimization metric reported to Ray Tune and Optuna.

4 Baseline Experiment (Part 1)

The baseline experiment trains the Transformer **exactly as-is** (i.e., without changing the original architecture or hyperparameters) for 100 epochs and evaluates BLEU using NLTK’s `corpus_bleu` with smoothing.

4.1 Baseline performance

From the baseline logs in `results.txt`, the final training loss and BLEU are:

4.2 Saved baseline artifact

The baseline notebook/script saves the final checkpoint as `transformer_translation_final.pth`, along with the vocabulary pickles `en_vocab.pkl` and `hi_vocab.pkl`, for later reuse in evaluation and tuned experiments.

Metric	Value
Total wall-clock time for 100 epochs	≈ 1 h 58 m
Final training loss (epoch 100)	0.0973
BLEU score (NLTK)	49.53

Table 1: Baseline metrics and recorded training runtime estimate (from per-epoch progress timing).

4.3 Evaluation protocol

The BLEU score is computed using NLTK’s `corpus_bleu` over a fixed set of example sentence pairs (the validation set used in the code). Tokenization is performed at the word level by splitting on whitespace. To reduce zero-count issues typical in early-stage generation, smoothing is applied using NLTK’s smoothing method4. Decoding at inference time uses greedy autoregressive generation to produce the predicted Hindi sequence.

4.4 Qualitative evaluation (baseline)

The baseline evaluation is performed on a small set of example sentences using greedy decoding and the produced translations are printed during execution.

5 Hyperparameter Tuning with Ray Tune + Optuna (Part 2)

The assignment requires refactoring the training loop such that Ray Tune can control and observe training progress. The implementation is contained in `b22cs001_ass_4_tuned_en_to_hi.py`.

5.1 Ray Tune compatible training function

The training loop is refactored into a Ray Tune entrypoint (e.g., `train_tune(config)`), which:

- Accepts a configuration dictionary `config`
- Initializes the Transformer, optimizer, and loss function using config values
- Reports intermediate training loss each epoch to Ray using `ray.train.report({"loss": epoch_loss})`; the standard training print/checkpoint behavior is replaced in the Ray worker by this metric reporting

5.2 Search space

Ray Tune defines a search space over at least four hyperparameters. Based on the implementation and tuning configuration recorded in `results.txt`, the tuned variables include:

- **Learning rate** (`lr`): log-uniform in $[10^{-5}, 10^{-3}]$
- **Batch size** (`batch_size`): choice $\{16, 32, 64\}$
- **Number of attention heads** (`num_heads`): choice $\{4, 8\}$ with `d_model=512` ensuring divisibility
- **Feed-forward dimension** (`d_ff`): choice $\{1024, 2048\}$
- **Dropout** (`dropout`): uniform in $[0.1, 0.4]$

5.3 Optuna search algorithm

Optuna is used as the search algorithm with an objective of minimizing training loss (metric `loss`, mode `min`):

`min loss.`

In practice, Ray Tune observes intermediate progress via the reported training loss at each epoch and uses this value as the optimization signal for Optuna and the ASHA scheduler.

The hyperparameter sweep is executed by instantiating a Ray Tune `Tuner` configured with an `OptunaSearch` instance and running `tuner.fit()`.

5.4 ASHA scheduling

To improve efficiency, ASHA is used as the early-stopping mechanism. In the observed run, each trial is capped at a maximum of `max_epochs=7`, and ASHA terminates poor trials early based on reported loss.

6 Efficiency Challenge (Part 3)

Training the Transformer for 100 epochs is computationally expensive. The tuning stage limits each trial to a maximum of `7 epochs` and uses ASHA to stop underperforming trials earlier.

6.1 Trial budget and sweep runtime

From the Ray Tune section in `results.txt`, the sweep contains 5 trials and completes in:

23 min 36 s.

6.2 Ray Tune trial summary

The table below summarizes the five trials reported in `results.txt`. Each trial is terminated either at the ASHA stopping point or at the trial cap of 7 epochs.

Trial	lr	batch	heads	d_ff	dropout	loss @ epoch
train_tune_2ac5ebd1	5.96869e-05	32	8	1024	0.373121	3.67058 @ 7
train_tune_03ea0d7b	1.49556e-04	16	4	1024	0.335692	2.49305 @ 7
train_tune_af5ff1f0	1.10794e-04	32	4	1024	0.182719	2.09566 @ 7
train_tune_68ab2bba	3.75870e-04	32	4	1024	0.185506	5.53649 @ 1
train_tune_26b82bf1	9.38233e-04	32	8	1024	0.157823	6.03208 @ 1

Table 2: Ray Tune trial outcomes (5 trials).

6.3 Best hyperparameters from Ray Tune

The best trial is:

- `lr = 1.10794e-04`
- `batch_size = 32`

- `num_heads = 4`
- `d_ff = 1024`
- `dropout = 0.182719`
- `max_epochs (trial cap) = 7`

7 Final Tuned Model Training and Evaluation

After selecting the best hyperparameters from Ray Tune, the model is retrained for **90 epochs** using the optimized configuration. This step is performed to validate that the found hyperparameters generalize beyond the short trial horizon.

7.1 Final tuned BLEU score

From `results.txt`, the tuned model achieves:

Metric	Value
Tuned model BLEU score (NLTK)	75.66

Table 3: Final tuned performance.

7.2 Improvement over baseline

The improvement relative to the baseline is:

$$75.66 - 49.53 = 26.13 \text{ BLEU points.}$$

This surpasses the assignment requirement of matching or exceeding baseline BLEU.

7.3 Qualitative translations (tuned model)

The tuned model outputs translations for representative English sentences as follows:

English input	Predicted Hindi output
I love you.	main tumse pyaar karta hoon.
What is your name?	tumhara naam kya hai?
How are you?	tu kaisi hai?
The weather is nice today.	aaj mausam achchha hai.
She is a good teacher.	vah ek achchha shikshak hai.

Table 4: Greedy-decoded qualitative examples from the tuned model.

8 Discussion

The results highlight two key findings:

- **Hyperparameter sensitivity:** Adjusting learning rate, batch size, attention configuration, feed-forward width, and dropout leads to substantial BLEU gains. The best configuration uses moderate attention complexity (4 heads) paired with `d_ff=1024` and dropout around 0.18.
- **Efficiency through ASHA:** Trials that performed poorly were terminated early (e.g., some trials ended at epoch 1), reducing compute consumption compared to fully training each candidate for the entire 100-epoch schedule.

Overall, the combined Optuna + ASHA strategy offers a principled approach to locate strong configurations quickly and then confirm them through longer retraining.

9 Conclusion

This work successfully demonstrates efficient hyperparameter optimization for Transformer-based English-to-Hindi translation. A baseline Transformer trained for 100 epochs achieves a BLEU score of 49.53. Using Ray Tune with Optuna and an ASHA scheduler, a set of hyperparameters is selected via training-loss minimization under a trial epoch cap of 7 epochs. Retraining with the best-found configuration yields a BLEU score of 75.66, which represents an improvement of 26.13 BLEU points over the baseline.

Reproducibility Note

The implementation and experimental workflow are provided in `b22cs001_ass_4_tuned_en_to_hi.ipynb`. All reported metrics and configurations are extracted from `b22cs001_ass_4_tuned_en_to_hi.ipynb`.